

Módulo Departments: Guía y Funcionamiento Técnico

Documentación integral del subdominio de Departamentos • Java 21 & Spring Boot 4

1. ¿Qué es y cuál es el rol del módulo Departments?

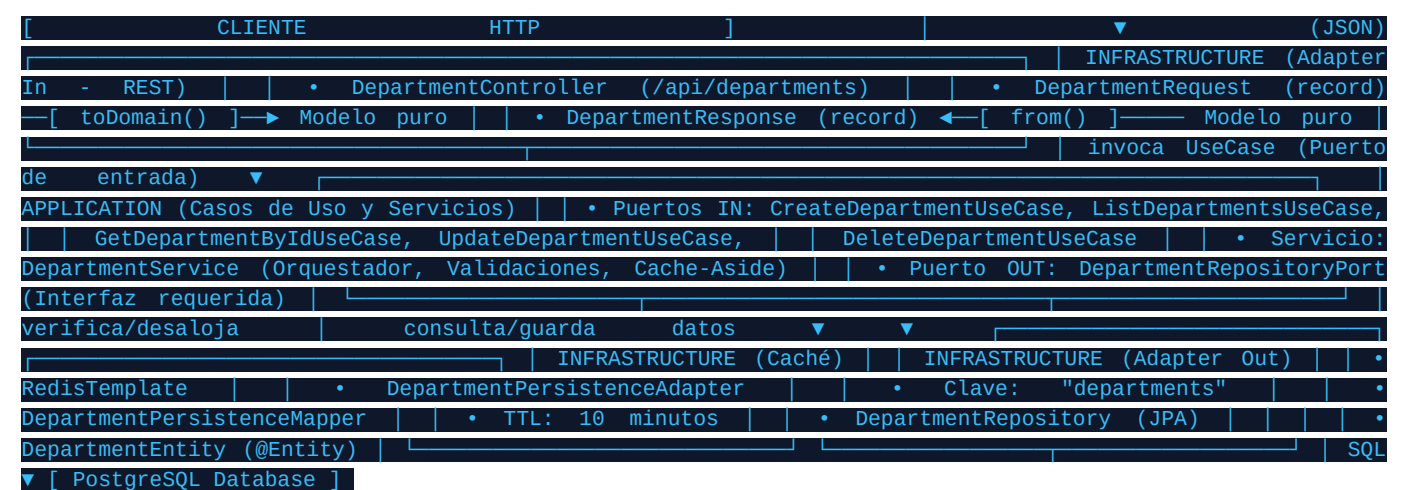
El módulo `departments` gestiona los departamentos organizacionales (tales como *Soporte TI*, *Recursos Humanos*, *Operaciones*, *Finanzas*) dentro del sistema Helpdesk. Representa una entidad troncal del sistema: los usuarios pertenecen a un departamento y los tickets de servicio se enrutan a través de ellos para definir prioridades y acuerdos de nivel de servicio (SLA).

Responsabilidades principales:

- Operaciones CRUD Completas:** Registro, listado general, búsqueda por identificador (UUID), edición de datos y eliminación.
- Reglas e Invariantes de Negocio:** Validación estricta de nombres no vacíos y verificación de no duplicidad de nombres antes de guardar.
- Alto Rendimiento con Redis:** Implementación del patrón *Cache-Aside* con tiempo de expiración (TTL de 10 min) y desalojo inmediato (*Eviction*) ante escrituras.
- Desacoplamiento Absoluto:** Modelo de dominio libre de dependencias de frameworks, bases de datos o anotaciones JPA.

2. Estructura y Arquitectura Hexagonal

El módulo aplica el patrón de **Puertos y Adaptadores**, asegurando que la lógica de negocio resida en el centro:



3. Detalle de Capas y Componentes

A. Capa de Dominio (`domain/`) - El Núcleo Puro

- **`Department.java`**: Modelo puro de Java con campos `id` (UUID), `name` (String), `active` (Boolean), `createdAt` y `updatedAt`. No usa anotaciones de persistencia ni librerías externas más allá de Lombok.
- **`DepartmentNotFoundException.java`**: Excepción de dominio cuando no se encuentra un departamento solicitado (mapeada a HTTP 404).
- **`DepartmentAlreadyExistsException.java`**: Excepción lanzada si el nombre del departamento ya existe (mapeada a HTTP 409 Conflict).

B. Capa de Aplicación (`application/`) - Casos de Uso

- **Puertos de Entrada (`port/in/`)**: `CreateDepartmentUseCase`, `ListDepartmentsUseCase`, `GetDepartmentByIdUseCase`, `UpdateDepartmentUseCase` y `DeleteDepartmentUseCase`. Cada uno expone una acción precisa.
- **Puerto de Salida (`port/out/DepartmentRepositoryPort.java`)**: Define el contrato que la infraestructura de persistencia debe cumplir (`save`, `findById`, `findAll`, `existsByName`, `existsById`, `delete`).
- **Servicio (`service/DepartmentService.java`)**: Implementa los 5 casos de uso. Ejecuta las validaciones de negocio, coordina las consultas a la base de datos e implementa la estrategia de lectura y desalojo de caché en Redis.

C. Capa de Infraestructura (`infrastructure/`) - Adaptadores

- **Adaptador REST (`DepartmentController.java`)**: Expone los endpoints REST, valida la entrada con Jakarta Validation y retorna códigos HTTP semánticos.
- **DTOs Inmutables**: `DepartmentRequest` cuenta con el método `toDomain()` para convertir el payload a modelo de dominio; `DepartmentResponse` cuenta con `DepartmentResponse.from(domain)` para enviar la respuesta JSON al cliente.
- **Adaptador de Persistencia (`DepartmentPersistenceAdapter.java`)**: Implementa `DepartmentRepositoryPort` comunicándose con PostgreSQL a través de `DepartmentRepository` (Spring Data JPA) y `DepartmentPersistenceMapper`.

4. Estrategia de Caché Distribuida con Redis

Para optimizar las lecturas frecuentes y proteger la base de datos relacional, el servicio aplica el patrón **Cache-Aside**:

- | 1. | Lectura | con | Cache-Aside | (GET): |
|----|---|-----|-------------|--------|
| | Al invocar <code>listDepartments()</code> , el servicio verifica primero si la clave <code>departments</code> existe en Redis. Si hay Cache Hit , retorna los datos en memoria en milisegundos. Si hay Cache Miss (o falla Redis), consulta PostgreSQL, almacena el resultado en Redis con un TTL de 10 minutos y responde al usuario. | | | |

2. Desalojo Inmediato (Cache Eviction):
Al realizar operaciones que alteren el estado (**POST / create**, **PUT / update** o **DELETE / delete**), el servicio ejecuta `evictDepartmentsCache()` para borrar la clave `departments` en Redis, garantizando consistencia absoluta en lecturas posteriores.

5. Catálogo de Endpoints REST

Método	Endpoint	Descripción	Código HTTP	Efecto en Caché
POST	/api/departments	Crea un nuevo departamento	201 Created	Invalida clave <code>departments</code>
GET	/api/departments	Lista departamentos (con caché)	200 OK	Lee / Pueba caché (TTL 10 min)
GET	/api/departments/{id}	Obtiene un departamento por ID	200 OK	Consulta directa a BD
PUT	/api/departments/{id}	Actualiza datos del departamento	202 Accepted	Invalida clave <code>departments</code>
DELETE	/api/departments/{id}	Elimina un departamento por ID	204 No Content	Invalida clave <code>departments</code>

Ejemplos de Contratos JSON:

1. Solicitud de Creación / Actualización (Request):

```
{
  "name": "Infraestructura & Servidores",
  "active": true
}
```

2. Respuesta Exitosa (Response):

```
{
  "id": "e3b0c442-98fc-1c14-9af0-2a3b4c5d6e7f",
  "name": "Infraestructura & Servidores",
  "active": true,
  "createdAt": "2026-08-28T14:30:00",
  "updatedAt": "2026-08-28T14:30:00"
}
```

3. Respuesta de Error (404 Not Found):

```
{
  "timestamp": "2026-08-28T14:35:10",
  "status": 404,
  "error": "Not Found",
  "message": "Department not found with id: e3b0c442-98fc-1c14-9af0-2a3b4c5d6e7f",
  "path": "/api/departments/e3b0c442-98fc-1c14-9af0-2a3b4c5d6e7f"
}
```

6. Buenas Prácticas y Patrones Destacados

- **Inmutabilidad y DTOs:** Uso de Java `record` para garantizar la inmutabilidad de los datos transferidos.
- **Mapeo Encapsulado:** Los métodos `toDomain()` y `from()` mantienen limpios los controladores sin mezclar lógica de construcción.
- **Resiliencia en Caché:** Si Redis no responde, el servicio captura la excepción e interactúa con PostgreSQL de forma transparente (*graceful fallback*).
- **Manejo Global de Excepciones:** Toda excepción de dominio es interceptada por `GlobalExceptionHandler` para devolver respuestas JSON estandarizadas.